

Relatório Técnico Final — FieldFlow

Projeto: FieldFlow — Marketplace de Serviços Agrícolas (AgTech) **Aluno:** Rafael Ganascini de Moura
Disciplina: Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas (LDAMD) **Instituição:** PUC Minas — Engenharia de Software · 1º Semestre de 2026 **Data:** 29 de junho de 2026

1. Introdução e contexto

O sucesso da produção agrícola depende do cumprimento rigoroso de janelas biológicas e climáticas. Produtores rurais frequentemente dependem de terceiros para realizar intervenções críticas — pulverização, colheita, análise de solo, reparos emergenciais —, mas enfrentam dificuldade em localizar prestadores disponíveis nos momentos de pico da safra. O **FieldFlow** ataca esse gargalo centralizando a demanda e otimizando a logística do prestador através de uma plataforma distribuída: o produtor publica uma demanda de campo e o prestador qualificado a visualiza, candidata-se e a executa.

O sistema atende dois perfis de usuário com fluxos distintos:

- **Cliente** (produtor rural): cria demandas, recebe e aceita propostas, acompanha a execução e avalia o prestador ao final.
- **Prestador** (técnico/operador): submete um currículo para aprovação, visualiza demandas pendentes, candidata-se e executa o serviço.

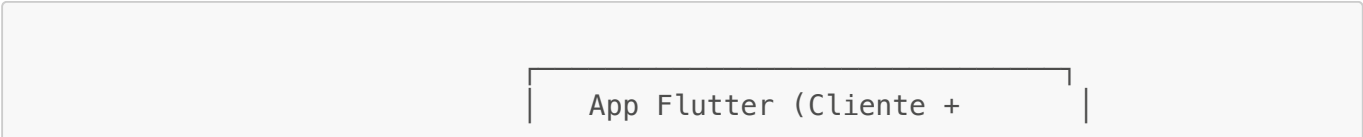
Este relatório descreve a arquitetura efetivamente implementada ao longo das quatro sprints do projeto integrador, as decisões de design que a moldaram, as dificuldades técnicas encontradas com suas respectivas soluções, e uma reflexão crítica sobre os quatro padrões arquiteturais estudados na disciplina: **Event-Driven Architecture (EDA)**, **Message-Oriented Middleware (MOM)**, **Clean Architecture** e **REST**.

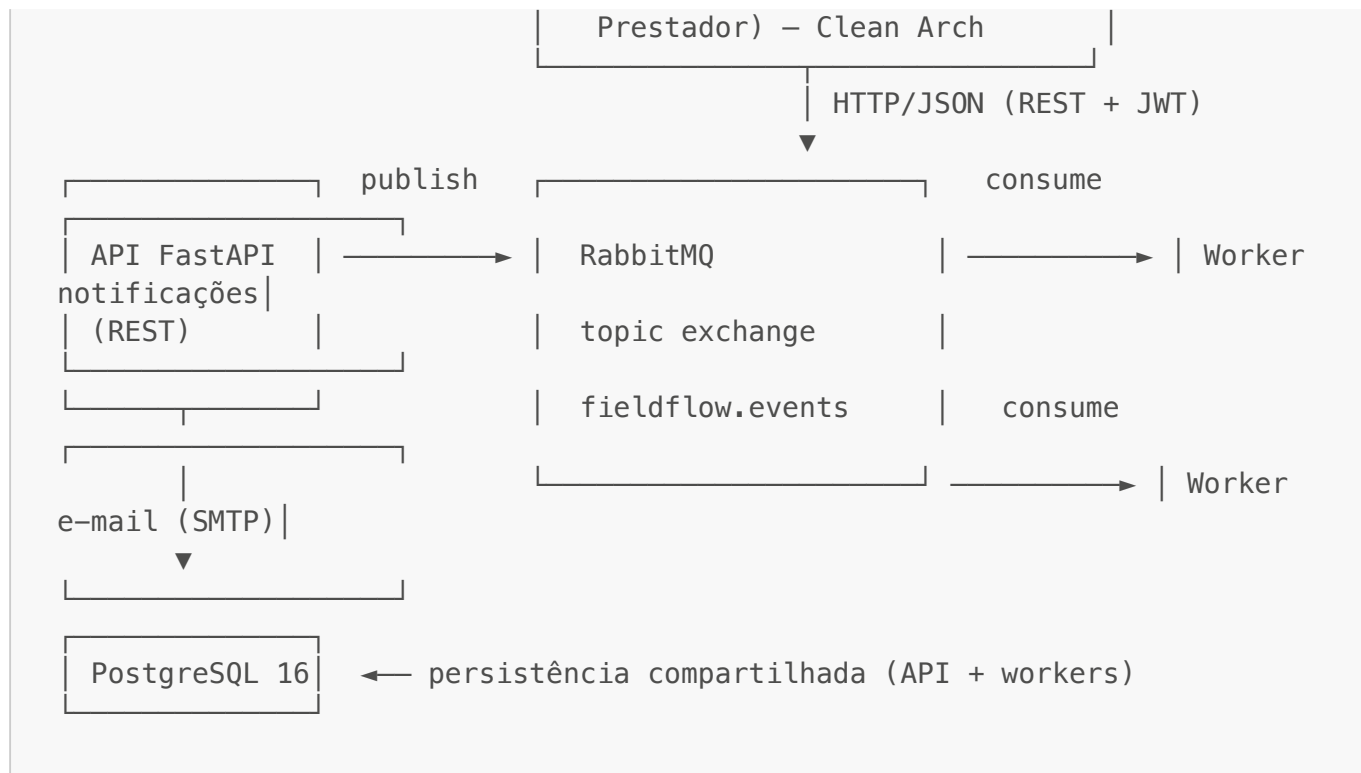
1.1 Evolução por sprints

Sprint	Foco	Entregue
1	Backend REST + modelagem (Clean Architecture por bounded context)	11/05/2026
2	Integração MOM (RabbitMQ) + comunicação assíncrona	25/05/2026
3	App Flutter — Cliente (consumo da API, mapas, avaliações)	15/06/2026
4	App Flutter — Prestador + entrega final	03/07/2026

2. Arquitetura implementada

O FieldFlow é um sistema distribuído composto por **quatro processos cooperantes**, orquestrados via Docker Compose, mais um aplicativo móvel cliente:





2.1 Backend — API REST (FastAPI)

O backend expõe uma **API REST** em FastAPI organizada em **Clean Architecture por bounded context**. Cada contexto de negócio é um módulo independente que replica as quatro camadas concêntricas da Clean Architecture:

```

codigo/Back/<contexto>/
├── domain/          # entidades e regras de negócio puras (sem framework)
├── application/     # casos de uso (orquestração); recebe dependências por
injeção
├── infrastructure/  # SQLAlchemy models + repositories (implementam
interfaces do domínio)
└── presentation/    # schemas Pydantic + routers FastAPI (entrada/saída
HTTP)

```

Os contextos implementados são: **auth**, **usuarios**, **demandas**, **candidaturas**, **prestadores**, **avaliacoes**, **notificacoes** e **emails**. Módulos transversais de infraestrutura completam o desenho: **core/** (configuração via *pydantic-settings* e engine/sessão SQLAlchemy), **mom/** (publisher de eventos), **notifier/** (envio SMTP) e **worker/** (consumidores assíncronos).

A **regra de dependência** da Clean Architecture é respeitada: o domínio não importa SQLAlchemy nem FastAPI; o caso de uso depende de **interfaces** de repositório (abstrações), e a implementação concreta com SQLAlchemy é injetada na camada de apresentação. Isso permitiu, por exemplo, testar os casos de uso de demandas e candidaturas com SQLite em memória e um **FakeEventPublisher**, sem subir PostgreSQL nem RabbitMQ.

2.2 Mensageria — EDA sobre MOM (RabbitMQ)

A comunicação assíncrona é o coração do sistema. A API **nunca processa diretamente** os efeitos colaterais de uma operação de negócio: ela apenas **publica um evento** no *topic exchange* `fieldflow.events` e responde ao cliente HTTP imediatamente. As chaves de roteamento seguem o formato `<recurso>.<acao>[.<detalhe>]`, por exemplo `demanda.status.aceito` ou `candidatura.aceita`.

Eventos publicados ao longo do ciclo de vida do sistema:

- `demanda.criada` / `demanda.atualizada` / `demanda.removida` / `demanda.status.<status>`
- `usuario.criado` / `usuario.atualizado` / `usuario.removido`
- `prestador.perfil.enviado` (API) / `prestador.aprovado` / `prestador.reprovado` (worker)
- `candidatura.criada` / `candidatura.aceita` / `candidatura.cancelada` / `candidatura.rejeitada`

Dois **consumidores independentes** se inscrevem nesses eventos, demonstrando o desacoplamento Pub/Sub na prática:

1. **Worker de notificações** — consome `demanda.#`, `usuario.#`, `prestador.#`, `candidatura.#` e grava um registro na tabela `notificacoes` (que alimenta o feed in-app). Também executa **lógica de negócio reativa**: ao receber `prestador.perfil.enviado`, valida o currículo (≥ 1 ano de experiência e ≥ 1 certificação) e publica `prestador.aprovado`/`prestador.reprovado`; ao receber `candidatura.aceita`, marca as candidaturas concorrentes da mesma demanda como **REJEITADA** em cascata (um evento dispara N reações).
2. **Worker de e-mail** — consome os mesmos eventos de forma totalmente independente e envia notificações por SMTP. Adicionar este segundo consumidor **não exigiu alteração alguma no produtor**, evidenciando o valor do desacoplamento Pub/Sub.

2.3 Mobile — App Flutter (Clean Architecture + MVVM)

O aplicativo móvel evoluiu de uma estrutura MVVM simples (Sprint 3) para uma **Clean Architecture completa** (Sprint 4), espelhando a disciplina de camadas do backend:

```
codigo/Mobile/field_flow/lib/
├── core/           # ApiClient (Facade sobre package:http), Config,
                    ApiException, tema, formatters
├── domain/         # entities (DTOs imutáveis), repositories (interfaces),
                    usecases
├── data/           # models (fromJson), datasources (chamadas REST),
                    repositories (implementações)
├── presentation/   # screens (View), viewmodels (ChangeNotifier), widgets
                    reutilizáveis
└── state/          # AuthController – sessão JWT global (fonte única de
                    "estou logado?")
```

- A **View** (`screens/`) não contém regra de negócio: observa o ViewModel via `provider/context.watch` e delega ações.
- O **ViewModel** (`viewmodels/`) detém o estado da tela (loading/erro/dados) e a orquestração; não importa `material.dart` nem usa `BuildContext`.

- Os **use cases** consomem **interfaces de repositório** (**domain/repositories**), cuja implementação (**data/repositories**) delega a **datasources** REST. A inversão de dependência é idêntica à do backend.

A **atualização assíncrona de estado** no app é feita por **polling** com **Timer.periodic** dentro dos ViewModels (intervalos de 6–10 s conforme a tela). Assim, quando o backend muda o estado de uma demanda por reação a um evento (ex.: aceite move **PENDENTE** → **ACEITO** e rejeita as concorrentes em cascata), a UI reflete a mudança sem ação manual do usuário.

2.4 Persistência e infraestrutura

PostgreSQL 16 é o banco compartilhado. O schema é versionado com **Alembic** (revisões **0001–0005**), e tanto a API quanto os workers executam **alembic upgrade head** programaticamente no *startup* — eliminando a necessidade de recriar volumes a cada mudança de schema. Todo o ambiente sobe com um único **docker compose up**, com **healthcheck** e **depends_on: condition: service_healthy** garantindo a ordem de inicialização (DB → RabbitMQ → API/workers).

3. Decisões de design

Clean Architecture por bounded context, não por camada técnica. Em vez de pastas globais **controllers/**, **services/**, **models/**, optei por modularizar por domínio (**demandas/**, **candidaturas/...**), cada um com suas quatro camadas. Isso mantém a coesão alta (tudo de "demandas" num só lugar) e o acoplamento baixo entre contextos, facilitando a evolução incremental sprint a sprint.

RabbitMQ em vez de Kafka ou Redis Pub/Sub. Kafka foi descartado pela complexidade operacional desnecessária a um projeto acadêmico individual. Redis Pub/Sub é *fire-and-forget*, sem persistência — inaceitável para eventos como "demanda criada". O RabbitMQ entrega broker AMQP maduro, filas duráveis, mensagens persistentes, roteamento declarativo por *topic* e uma UI de gerenciamento que facilita evidenciar o funcionamento.

cliente_id derivado do JWT, nunca do corpo da requisição. Em rotas autenticadas, a identidade do autor vem sempre do **current_user** extraído do token, e não de um campo enviado pelo cliente — evitando que um usuário crie recursos em nome de outro.

Aceite de demanda restrito a um endpoint dedicado. A transição para **ACEITO** só ocorre via **POST /candidaturas/{id}/aceitar**; um **PATCH /demandas/{id}/status** com **ACEITO** retorna **409 Conflict**. Isso concentra a regra crítica (e a rejeição em cascata) num único ponto e evita estados inconsistentes.

Polling no app em vez de WebSockets/MOM no mobile. A atualização assíncrona da UI foi resolvida com polling sobre a API REST existente. WebSockets exigiriam um novo endpoint e infraestrutura de conexão persistente; o polling atende ao requisito de "reação sem ação manual" reutilizando o que já existia, com custo de implementação muito menor.

Idempotência por event_id. Cada **publish()** gera um UUID (**event_id**) injetado tanto no **message_id** da AMQP quanto no **payload**. A tabela **notificacoes** tem **UNIQUE(event_id)** e o consumidor descarta duplicatas — tornando o processamento idempotente diante de *redeliveries*.

4. Dificuldades encontradas e soluções adotadas

Ordem de inicialização dos containers. A API e os workers tentavam conectar ao RabbitMQ antes de ele estar pronto, derrubando os processos. *Solução:* `depends_on: condition: service_healthy` no Compose, somado a um `retry loop` no worker (`_connect_with_retry`, 30 tentativas × 2 s) para resiliência além do *startup*.

Acoplamento da mensageria nos testes. Os casos de uso publicam eventos, mas os testes não devem depender de um broker real. *Solução:* a interface `EventPublisher` tem uma implementação `NoopEventPublisher` selecionada em tempo de execução conforme a presença da variável `RABBITMQ_URL`, e um `FakeEventPublisher` nos testes para asserções sobre os eventos emitidos.

Entrega "pelo menos uma vez" e mensagens duplicadas. O modelo AMQP pode reentregar uma mensagem após falha de *ack*. *Solução:* idempotência por `event_id` com restrição `UNIQUE` no banco (descrita na seção 3).

Falhas no processamento dos eventos. Uma mensagem malformada ou um erro transitório no consumidor não podiam travar a fila nem ser perdidos silenciosamente. *Solução:* **Dead-Letter Queue** — exchange `fieldflow.events.dlx` (fanout) + fila `fieldflow.notificacoes.dlq`; falhas vão para a DLQ via `nack(requeue=False)`. O reprocessamento é manual (`python -m worker reprocess-dlq`): drena a DLQ, lê a *routing key* original do header `x-death` e republica no exchange principal, com a deduplicação garantindo que não haja efeito duplicado.

Evolução do schema sem perder dados. Nas primeiras sprints, mudar o schema exigia `docker compose down -v`, apagando o banco. *Solução:* adoção do **Alembic** com migrações versionadas e `upgrade head` automático no startup.

Consistência entre commit no banco e publicação do evento. Permanece como **dívida técnica assumida e documentada**: uma falha entre o `commit` no PostgreSQL e o `basic_publish` no RabbitMQ pode deixar o sistema inconsistente (a API responde sucesso, mas o evento nunca chega ao broker). A solução planejada é o **Outbox Pattern**: gravar o evento numa tabela `outbox` na mesma transação do agregado e ter um *relay* separado publicando a partir dela. Reconhecer e documentar a limitação foi a decisão consciente, dado o escopo do projeto.

Dados sensíveis em eventos. Senhas (em texto ou hash) **nunca** integram o *payload* de um evento, evitando vazamento de credenciais por um canal de mensageria que pode ser inspecionado.

5. Reflexão sobre os padrões estudados

5.1 Event-Driven Architecture (EDA)

A EDA mostrou-se o padrão mais transformador do projeto. Ao inverter o controle — o produtor emite um fato consumado ("a candidatura foi aceita") em vez de comandar um serviço ("rejeite as concorrentes") —, o sistema ganhou extensibilidade: o worker de e-mail foi acrescentado **sem tocar uma linha do produtor**. Conforme Hohpe e Woolf (2003), o desacoplamento temporal e referencial entre produtor e consumidor é justamente o que viabiliza essa evolução independente. O caso da **rejeição em cascata** é um exemplo *textbook* de "um evento dispara N reações", algo que numa orquestração síncrona obrigaria o cliente HTTP a esperar o processamento de cada rejeição. O custo do padrão é a **consistência eventual**: abandona-se a transação distribuída em favor de reações assíncronas, o que exige

idempotência, tratamento de falhas (DLQ) e — idealmente — o Outbox Pattern para garantir atomicidade entre persistência e publicação. Bellemare (2020) discute precisamente esses *trade-offs* ao tratar eventos como o contrato de integração entre serviços.

5.2 Message-Oriented Middleware (MOM)

O MOM (RabbitMQ) é a infraestrutura que torna a EDA confiável. Filas duráveis e mensagens persistentes garantem que um evento sobreviva à queda de um consumidor; o *topic exchange* permite roteamento declarativo por padrão de chave, de modo que cada consumidor declara apenas o que lhe interessa (*demanda.#*, *candidatura.#...*). O MOM proporciona o **desacoplamento temporal** — produtor e consumidor não precisam estar online simultaneamente — e o **balanceamento de carga natural** via *work queues*. A contrapartida é operacional: foi preciso lidar com ordem de inicialização, *retries*, *redelivery* e uma estratégia explícita para mensagens "venenosas" (DLQ). O MOM não elimina a complexidade dos sistemas distribuídos; ele a **move para pontos bem definidos e gerenciáveis**, como aponta Tanenbaum e Van Steen (2017) ao classificar a comunicação por mensagens como pilar dos sistemas distribuídos fracamente acoplados.

5.3 Clean Architecture

A Clean Architecture organizou tanto o backend quanto o app. Seu maior benefício concreto foi a **testabilidade**: como os casos de uso dependem de interfaces e não de SQLAlchemy ou FastAPI, foi possível testá-los com SQLite em memória e *fakes*, isolando regra de negócio de detalhe de infraestrutura. A **regra de dependência** de Martin (2017) — dependências apontando sempre para dentro, em direção às políticas de negócio — provou-se prática, não apenas teórica: trocar o publisher real por um *noop*, ou o banco real por um em memória, é uma decisão de injeção, não uma reescrita. O custo é a **verbosidade**: quatro camadas por contexto geram muito *boilerplate* (entidade, model, repositório, schema), o que para alguns CRUDs simples parece excessivo. A escolha de modularizar **por bounded context** mitigou isso, mantendo cada domínio coeso e navegável.

5.4 REST

O REST foi a fronteira entre o app móvel e o backend, e a base sobre a qual a EDA opera internamente. A modelagem por **recursos** (*/demandas*, */candidaturas*, */prestadores/{id}/perfil*) e o uso semântico dos **verbos HTTP** (*GET* para leitura, *POST* para criação, *PATCH* para transição de status, *PUT* para atualização) deram à API uma interface previsível e autodocumentável — reforçada pelo OpenAPI gerado automaticamente pelo FastAPI. Os **códigos de status** carregam significado de negócio (*409 Conflict* quando se tenta aceitar uma demanda pela rota errada). A natureza **stateless** do REST, com autenticação via Bearer JWT em cada requisição, simplificou o escalonamento e a divisão cliente/servidor. A principal limitação sentida foi a ausência de *push*: o REST é *pull* por natureza, e foi por isso que o app recorreu a **polling** para refletir mudanças de estado — uma solução pragmática, ainda que menos elegante que *server push*. Os princípios seguidos alinham-se à dissertação seminal de Fielding (2000), que define REST como um estilo arquitetural orientado a recursos, sem estado e com interface uniforme.

5.5 Síntese

Os quatro padrões não competem; **compõem-se em camadas**. O REST define a fronteira síncrona com o cliente; a Clean Architecture organiza o que acontece dentro de cada serviço; a EDA descreve como os

serviços reagem entre si; e o MOM é a infraestrutura que torna essas reações confiáveis e desacopladas. A maior lição do projeto foi perceber que a escolha não é "REST **ou** eventos", mas **REST na borda e eventos no interior** — síncrono onde o usuário espera resposta, assíncrono onde a reação pode (e deve) acontecer em segundo plano.

6. Conclusão

O FieldFlow entregou um sistema distribuído funcional e coeso: uma API REST em Clean Architecture, comunicação 100% assíncrona via EDA sobre RabbitMQ com dois consumidores independentes, e um aplicativo Flutter para os dois perfis de usuário, também em Clean Architecture. As principais decisões de design — modularização por bounded context, idempotência por `event_id`, DLQ para resiliência, JWT *stateless* e polling no cliente — mostraram-se adequadas ao escopo, e as dívidas técnicas remanescentes (sobretudo o Outbox Pattern) foram identificadas e documentadas em vez de ignoradas. Mais do que cumprir requisitos, o projeto consolidou o entendimento de que arquitetura distribuída é, antes de tudo, um exercício de **escolher onde colocar o acoplamento** — e os padrões EDA, MOM, Clean Architecture e REST são ferramentas complementares para fazer essa escolha de forma consciente.

7. Screencast (demonstração em vídeo)

A demonstração completa do FieldFlow em funcionamento — desde a subida do ambiente com Docker Compose até os fluxos de Cliente e Prestador no app Flutter, passando pela comunicação assíncrona via RabbitMQ — está disponível no YouTube:

 **Screencast:** <https://youtu.be/SOb5KJmm8G0>

Referências

BELLEMARE, Adam. **Building Event-Driven Microservices: Leveraging Organizational Data at Scale**. Sebastopol: O'Reilly Media, 2020.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado em Information and Computer Science) — University of California, Irvine, 2000.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions**. Boston: Addison-Wesley, 2003.

MARTIN, Robert C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Prentice Hall, 2017.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed Systems**. 3. ed. Pearson, 2017.